

INDIAN INSTITUTE OF TECHNOLOGY ROORKEE

DESIGN AND IMPLEMENTATION OF A

MEMORY MANAGEMENT SIMULATOR

Dynamic Memory Allocation and Cache Hierarchy Simulation

Submitted by: Saral Gupta
Enrollment No.: 24115130
Branch: Electrical Engineering

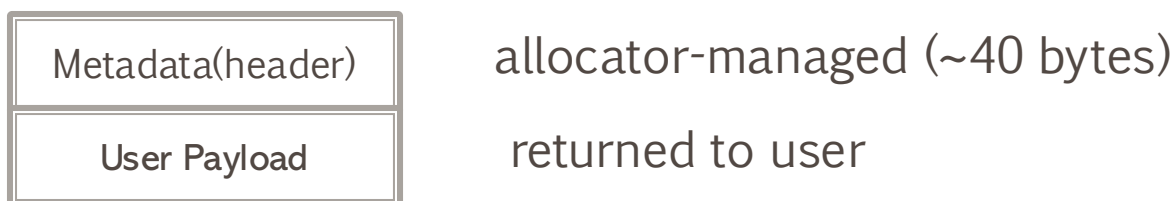
This project presents the design and implementation of a *memory management simulator supporting multiple allocation strategies, a buddy system allocator, and an optional multi-level cache hierarchy.*

Memory Layout and Assumptions

▪Memory Layout

- The memory management simulator operates on a **single contiguous heap region** allocated at initialization time using a simulated **sbrk** interface. This heap represents a simplified model of physical memory and is fully managed by the custom allocator without relying on the system allocator.
- Each allocated block in the heap follows a **header–payload layout**, where metadata is stored **inside the heap itself**, immediately preceding the user-accessible memory region.

General Block Structure



```
/* Block metadata stored INSIDE heap */
typedef struct metadata {
    size_t size;           // 8 bytes
    size_t requested_size; // 8 bytes
    int    in_use;         // 4 bytes (+padding)
    struct metadata *next; // 8 bytes
    struct metadata *prev; // 8 bytes
} metadata_t;
```

- The metadata structure stores information required for allocation, deallocation, and fragmentation analysis, including:
 - Total block size (metadata + payload)
 - User-requested payload size
 - Allocation status (free or in use)
 - Pointers for linked-list management (for non-buddy allocators)

As a result, **every allocation incurs a fixed metadata overhead** in addition to the requested payload size. In this implementation, the metadata occupies approximately **40 bytes per allocation**, which is deliberately exposed and tracked as part of the simulation.

Allocation Model Assumptions

▪The following assumptions were made to keep the simulator focused, deterministic, and easy to analyze:

1). Fixed Heap Size

- The heap size is specified once during initialization.
- The heap does not grow or shrink dynamically.
- All allocation failures occur due to fragmentation or insufficient free space.

2). In-Heap Metadata Overhead

- All allocators store metadata inside the heap.
- Each allocation consumes additional memory beyond the requested size (≈ 40 bytes).
- Internal fragmentation caused by metadata is explicitly visible and intentional.

3). Single-Threaded Execution

- The simulator assumes a single-threaded environment.
- No synchronization, locking, or concurrent access is supported.

4). No Virtual Memory Paging

- Virtual memory is simulated conceptually, but no real page tables or OS-backed paging mechanisms are implemented.
- All addresses used in the simulator directly map to offsets within the simulated heap.

5). Allocator Strategy Isolation

- First Fit, Best Fit, Worst Fit, and Buddy allocators are implemented as independent strategies.
- Only one allocator strategy is active at a time.
- The heap is reinitialized before switching strategies during testing.

6). Different Internal Data Structures

- List-based allocators (First Fit, Best Fit, Worst Fit) use a doubly linked list of blocks.
- The Buddy allocator uses power-of-two block sizes and per-order free lists.
- As a result, some observability tools (e.g., linear memory dump) are applicable only to list-based allocators.

6). Simplified Alignment

- Blocks are aligned naturally based on metadata size.
- No explicit cache-line or page-size alignment constraints are enforced beyond allocator logic.

Design Rationale

▪This design prioritizes **clarity and traceability** over realism. By placing metadata inside the heap and accounting for its size explicitly, the simulator makes fragmentation, allocator overhead, and memory efficiency **directly observable**. The assumptions intentionally simplify OS-level complexities so that allocator behavior and cache interactions can be studied in isolation.

Common Architecture for List-Based Allocation Strategies

(First Fit, Best Fit, Worst Fit)

- First Fit, Best Fit, and Worst Fit allocation strategies in this simulator are implemented within a **shared architectural framework**. These strategies differ only in **how a free block is selected**, while all other aspects of memory management—heap structure, metadata layout, block splitting, and coalescing—remain identical.
- This design ensures consistency across strategies and allows direct comparison of their allocation behavior under identical conditions.

Heap Representation and Metadata Structure

All three strategies operate on a **single contiguous heap** initialized once at program startup. The heap is managed as a **doubly linked list of memory blocks**, where each block consists of in-band metadata followed by the user-accessible payload.

Each block is represented by a **metadata_t** structure stored directly inside the heap. This metadata contains:

- Total block size (including metadata and payload)
- Requested payload size

Allocation status (**in_use**)

Pointers to the previous and next blocks in the list

Because metadata is stored in-band, each allocation incurs a **fixed overhead of approximately 40 bytes**, which is explicitly accounted for in all allocation decisions.

Shared Allocation Workflow

- All list-based allocation strategies follow the same high-level allocation workflow:

1). Allocation request

- The user calls **my_malloc(size)**.
- The allocator increments global allocation statistics.

2). Strategy dispatch

- The currently selected allocation strategy is stored in a global allocator mode.
- **my_malloc** dispatches the request to the appropriate strategy-specific function:

first_fit_malloc

best_fit_malloc

worst_fit_malloc

3). Block search

- The heap list is traversed from the beginning.
- Only free blocks with sufficient size are considered.
- The strategy determines which eligible block is selected.

4). Block splitting

- If the selected free block is significantly larger than required, it is split into:
 - an allocated block sized for the request
 - a remaining free block
- Splitting occurs only if the remainder can accommodate metadata and a minimum payload.

5). Allocation completion

- The selected block is marked as allocated.
- The requested size is recorded in metadata.
- A pointer to the payload region is returned to the user.

Shared Deallocation Workflow

- Deallocation is also common across all three strategies:

1). Free request

The user calls `my_free(ptr)`.

- The metadata corresponding to the payload pointer is located.

2). Mark block free

- The block is marked as not in use.
- The requested size is reset.

3). Immediate coalescing

- If the adjacent block on either side is free, the blocks are merged.
- Coalescing is performed eagerly to reduce external fragmentation.

This unified deallocation logic ensures consistent heap recovery behavior regardless of which list-based strategy was used for allocation.

Strategy Selection Mechanism

- The allocator supports dynamic selection of allocation strategies through a global strategy flag. The active strategy can be changed at runtime (e.g., via the CLI), but only **one strategy is active at any given time**.
- Internally:
 - The heap structure remains unchanged.
 - Only the block selection policy differs.
 - Allocation and deallocation logic remain consistent.
- This separation of **policy (selection strategy)** from **mechanism (heap management)** simplifies the implementation and highlights the behavioral differences between strategies.

First Fit Allocation Strategy

Detailed Allocation Procedure

- When a memory allocation request is made, the following steps are performed:

1). Request Handling

The allocation request is received through `my_malloc(size)`.

- Control is dispatched to `first_fit_malloc(size)` based on the active strategy.

2). Total Size Computation

- The allocator computes the total required block size as:

`total_size = sizeof(metadata_t) + requested_size`

- This ensures that metadata overhead is explicitly accounted for.

3). Linear Heap Traversal

- The heap is represented as a doubly linked list of blocks.

Traversal begins from the head of the list (`heap_head`).

- Each block is checked for:

- Free status
- Sufficient size

4). Block Selection

- The **first free block** with size $\geq$ total required size is selected.
- No further blocks are examined after a suitable block is found.

5). Block Splitting

- If the selected block is larger than required, it is split into:
 - An allocated block of the required size
 - A remaining free block
- Splitting is performed only if the remaining portion can hold:
 - Metadata
 - A minimum payload size

6). Allocation Finalization

- The selected block is marked as allocated.
- The requested payload size is stored in metadata.
- A pointer to the payload region is returned to the user.

Deallocation and Coalescing

- Deallocation follows a **strategy-independent** process shared by all list-based allocators:

1. The metadata corresponding to the payload pointer is located.
2. The block is marked as free.
3. **Immediate coalescing** is performed:
 - If the next block is free, it is merged.
 - If the previous block is free, it is merged.
4. This eager merging reduces external fragmentation and restores larger free blocks.

Fragmentation Characteristics

▪ Advantages

- Fast allocation due to early termination of heap traversal.
- Low computational overhead.

▪ Disadvantages

- Can lead to **external fragmentation**, as small unusable free blocks accumulate near the beginning of the heap.
- Allocation quality depends heavily on allocation order.

Best Fit Allocation Strategy

- The **Best Fit** allocation strategy selects the **smallest free memory block** that is large enough to satisfy a given allocation request. Unlike First Fit, which stops at the first suitable block, Best Fit examines **all free blocks** in the heap before making a decision.
- The primary objective of this strategy is to **minimize wasted space** within allocated blocks and reduce external fragmentation.

Detailed Allocation Procedure

- Best Fit follows the same overall allocation framework as other list-based allocators, with a key difference in **block selection policy**:

1). Request Handling

- Allocation requests are dispatched from `my_malloc(size)` to `best_fit_malloc(size)` when the Best Fit strategy is active.

2). Total Size Computation

- The allocator computes the total required block size, including metadata overhead:

`total_size = sizeof(metadata_t) + requested_size`

3). Complete Heap Traversal

- The heap is traversed from beginning to end.
- All free blocks with sufficient size are considered as candidates.

4). Best Block Selection

- Among all suitable free blocks, the block with the **smallest size** is selected.
- This ensures that excess unused space within the allocated block is minimized.

5). Block Splitting

- If the selected block is larger than required, it is split into:
 - an allocated block of the required size
 - a remaining free block
- Splitting is performed only if the remaining space can form a valid free block.

6). Allocation Finalization

- The selected block is marked as allocated.
- The requested payload size is stored in metadata.
- A pointer to the payload region is returned to the user.

Deallocation and Coalescing

1. Deallocation behavior in Best Fit is **identical** to First Fit and other list-based allocators:
2. Freed blocks are immediately marked as available.
3. Adjacent free blocks are merged eagerly.
4. This coalescing step helps reduce long-term external fragmentation.

Fragmentation Characteristics

▪ Advantages

- Reduces internal wastage within allocated blocks.
- More space-efficient than First Fit in many scenarios.

▪ Disadvantages

- Requires full heap traversal for each allocation.
- Higher allocation time complexity.
- Can produce many small free blocks that are difficult to reuse.

Worst Fit Allocation Strategy

- The **Worst Fit** allocation strategy selects the **largest available free memory block** that can satisfy a given allocation request. Unlike First Fit and Best Fit, which prioritize speed and space efficiency respectively, Worst Fit attempts to avoid the creation of very small unusable fragments by allocating from the largest block.
- This strategy is intended to preserve medium-sized free blocks at the cost of potentially higher fragmentation over time.

Detailed Allocation Procedure

- Worst Fit follows the same common allocation framework used by other list-based strategies, with a distinct block selection policy:

1). Request Handling

- Allocation requests are dispatched from `my_malloc(size)` to `worst_fit_malloc(size)` when the Worst Fit strategy is active.

2). Total Size Computation

- The total required size is computed as:

$$\text{total_size} = \text{sizeof}(\text{metadata_t}) + \text{requested_size}$$

- This ensures metadata overhead is included in the allocation decision.

3). Complete Heap Traversal

- The entire heap is traversed from the head of the block list.
- All free blocks large enough to satisfy the request are examined.

4). Worst Block Selection

- Among all suitable free blocks, the block with the **largest size** is selected.
- This selection aims to minimize the creation of very small leftover fragments.

5). Block Splitting

- If the selected block is significantly larger than required, it is split into:
 - an allocated block of the required size
 - a remaining free block
- Splitting occurs only when the remaining portion can accommodate metadata and a minimum payload.

6). Allocation Finalization

- The selected block is marked as allocated.
- The requested payload size is recorded in metadata.
- A pointer to the payload region is returned to the user.

Deallocation and Coalescing

1. Worst Fit uses the **same deallocation and coalescing logic** as other list-based allocators:
2. Freed blocks are marked as available.
3. Adjacent free blocks are immediately merged.
4. Coalescing is delegated to the shared free routine to maintain consistency across strategies.

Fragmentation Characteristics

▪ Advantages

- Reduces the likelihood of producing very small free blocks.
- Preserves medium-sized blocks for future allocations.

▪ Disadvantages

- Often leads to **poor overall space utilization**.
- Can increase external fragmentation over time.
- Requires full heap traversal, increasing allocation latency.
- May negatively impact cache locality due to allocation from large blocks.

Buddy Allocation Strategy

- The Buddy Allocation strategy is designed to provide **fast allocation and deallocation** with efficient coalescing by restricting memory blocks to **power-of-two sizes**. Unlike list-based allocators, which rely on sequential traversal of the heap, the Buddy system uses structured splitting and merging rules that allow constant-time determination of adjacent free blocks.
- This allocator is particularly effective in scenarios where **frequent allocations and deallocations** occur, and predictable performance is more important than minimizing internal fragmentation.

Memory Organization and Block Orders

- In the Buddy system, the heap is divided into blocks whose sizes are powers of two:
 $2^{MIN_ORDER}, 2^{MIN_ORDER+1}, \dots, 2^{MAX_ORDERS-1}$

Each block size is referred to as an **order**.

For example:

- Order 7 → block size = 128 bytes
- Order 8 → block size = 256 bytes
- At initialization, the heap is represented as a **single free block** at the largest supported order. All smaller blocks are created dynamically through recursive splitting.
- Each order maintains its own **free list**, allowing fast lookup of available blocks.

Allocation Mechanism

When a memory allocation request is issued:

1). Size Normalization

- The requested size is increased to account for metadata overhead.
- The smallest block order capable of holding this total size is computed.

2). Free List Search

- The allocator searches the free list corresponding to the target order.
- If no block is available, higher-order free lists are examined.

3). Recursive Splitting

- If a larger block is found, it is repeatedly split into two equal halves.
- One half is retained for further splitting or allocation.
- The other half (its *buddy*) is returned to the appropriate free list.

4). Block Assignment

- Once a block of the target order is obtained, it is marked as allocated.
- The allocator returns a pointer to the payload region following metadata.

This approach ensures allocation time remains bounded and does not depend on heap size.

Deallocation and Buddy Coalescing

- Deallocation in the Buddy system is symmetric to allocation and is designed to be highly efficient.

When a block is freed:

- 1). The block is marked as free.
- 2). Its **buddy address** is computed using an XOR operation on the block's offset.
- 3). If the buddy:
 - is free
 - has the same block size
 - lies within heap boundsthen the two blocks are merged into a larger block.
- 4). This merging process continues recursively until:
 - the buddy is allocated, or
 - the maximum block order is reached.

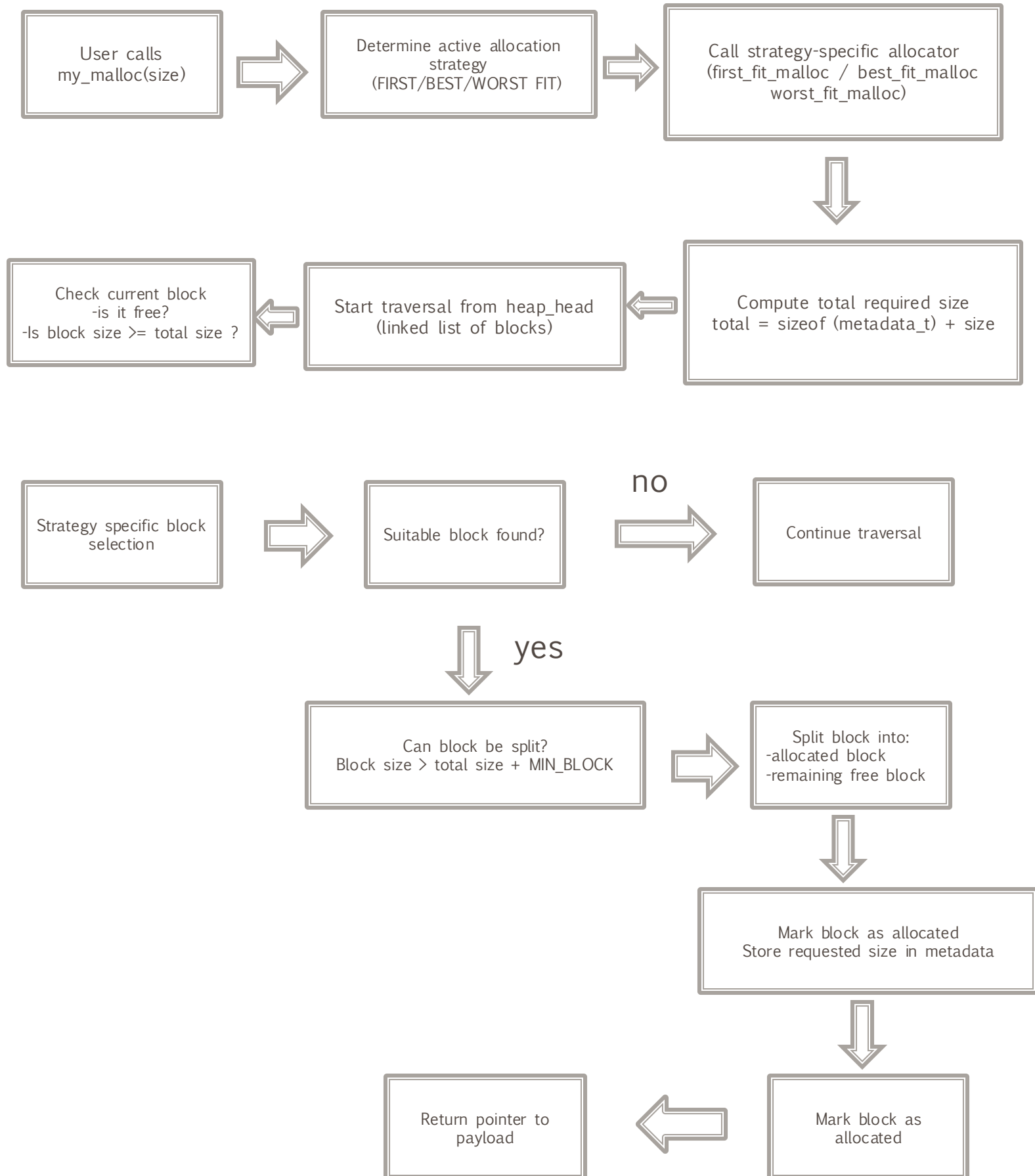
This deterministic merging mechanism avoids the need for linear heap traversal and guarantees fast coalescing.

Architectural Separation from List-Based Allocators

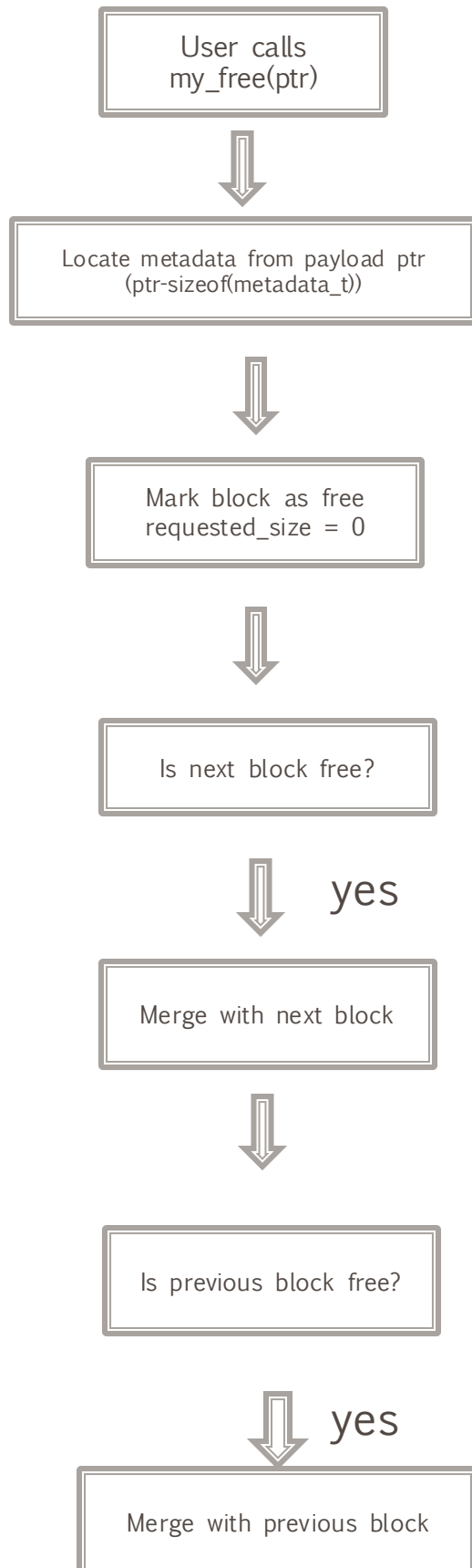
- The Buddy allocator is implemented as a **logically independent strategy** within the simulator.
- It does **not** use the linked-list heap structure.
- It maintains its own internal free lists per order.
- Observability tools designed for list-based allocators (e.g., linear memory dumps) are not directly applicable.

This separation highlights the fundamental architectural difference between sequential-fit allocators and structured allocators.

Allocation Flow Chart



Deallocation Flow Chart



Cache Hierarchy and Replacement Policy

Cache Initialization and Configuration

- The cache subsystem in the memory simulator is **explicitly initialized in `main.c`** and is designed to be **optional**. Cache initialization occurs **once at program startup**, before any memory allocation or CLI interaction begins.
- When caching is enabled, three cache levels—L1, L2, and L3—are instantiated as global objects and initialized using the **`cache_init`** function.

```
#ifdef ENABLE_CACHE

    cache_init(&L1, 1024, 64, 4); // 16 lines, 4-way
    cache_init(&L2, 2048, 64, 4); // 32 lines, 4-way
    cache_init(&L3, 4096, 64, 8); // 64 lines, 8-way

#endif
```

Cache Configuration Parameters

- Each cache level is initialized using:

```
cache_init(CacheLevel *cache,
           uint32_t cache_size,
           uint32_t block_size,
           uint32_t associativity);
```

Parameter	Description
cache_size	Total cache capacity in bytes
block_size	Size of one cache block (bytes)
associativity	Number of lines per set (N-way set associative)

The number of sets is computed internally as:

$$\text{num_sets} = \text{cache_size} / (\text{block_size} \times \text{associativity})$$

Cache Access Flow (Hierarchy)

All memory accesses go through the function:

```
cache_hierarchy_access(&L1, &L2, &L3, address);
```

Access Sequence

- **Check L1**
If hit → stop
- **Check L2**
If hit → insert into L1
- **Check L3**
If hit → insert into L2 and L1
- **Miss in all levels**
Simulate memory fetch
Insert into L3 → L2 → L1

This strictly enforces **inclusive cache behavior**.

Address Breakdown and Mapping

Address decoding is handled in `cache.c`:

```
AddressFields extract_address_fields(CacheLevel *cache, uint64_t address);
```

Each address is split into:

- **Offset** → determined by block size
- **Set Index** → determined by number of sets
- **Tag** → remaining higher bits

This allows correct simulation of **real hardware cache indexing**.

Replacement Policy (LRU & FIFO)

Policies Implemented

- **LRU (Least Recently Used):** Primary policy for L1/L2 to maximize hit rates.
- **FIFO (First-In-First-Out):** Supported for Lower Level Caches (L3) for simplicity.

How It Works

- **Dual Tracking:** Each cache line maintains both an `insertion_time` and an `lru_time`.
- **Atomic Updates:** On a **Hit**, the `lru_time` is updated to the current global clock (refreshing the line's "age").
- **Eviction:** When a set is full, the policy selects the victim with the oldest timestamp.

Why LRU?

- **Temporal Locality:** Ensures frequently accessed data remains in the cache.
- **Stability:** Prevents "thrashing" where useful data is evicted too early (fixing the L2 bypass issue).
- **Accuracy:** Atomic hit/miss logic prevents "ghost" data entries.

Cache Statistics and Observability

- Each cache level maintains:
- **Hit count** (Requests served immediately)
- **Miss count** (Requests forwarded to next level)
- **Hit Ratio** (Efficiency metric)
- **Policy Type** (Displays whether LRU or FIFO is active)
- Statistics are printed using `cache_print_stats`.

```
cache_print_stats("L1", &L1);
cache_print_stats("L2", &L2);
cache_print_stats("L3", &L3);
```

Enabling and Disabling Cache (IMPORTANT)

- The cache system is optional and can be controlled in two ways:

1). Compile-Time Control (Command Line)

Enable Cache

Delete the existing `memsim.exe` file and then

Run command

```
mingw32-make -B CFLAGS="-Wall -g -DENABLE_CACHE"
```

Then run:

```
.\memsim.exe --cli
```

This:

- Defines `ENABLE_CACHE`
- Compiles cache code
- Enables cache initialization and access

2). Manual Control (Code-Level)

- Cache logic is wrapped inside preprocessor guards:

```
#ifdef ENABLE_CACHE
    cache_hierarchy_access(&L1, &L2, &L3, address);
#endif
```

- To manually disable caching:
- Either remove `-DENABLE_CACHE` from build flags
- Or comment out cache calls in `main.c`
- This allows:
 - Running allocator-only experiments
 - Isolating allocator behavior from cache effects

Disable Cache

Run command

```
mingw32-make -B CFLAGS="-Wall -g"
```

Then run normally:

```
.\memsim.exe --cli
```

This:

- Excludes all cache-related logic
- Cache code is not executed at runtime

Command-Line Interface (CLI)

Running the Simulator in CLI Mode

- To use the interactive command-line interface, the simulator must be executed with the `--cli` flag:

```
.\memsim.exe --cli
```

- This launches an interactive prompt that allows users to manually issue memory management commands. When the flag is omitted, the simulator executes predefined test cases from `main.c`.

Memory Initialization

```
init memory <size>
```

- Initializes the heap with the specified size (in bytes). This command must be executed before performing any allocation. It resets the allocator state and prepares the heap for subsequent operations.

Allocation Strategy Selection

```
strategy <first | best | worst | buddy>
```

- Selects the memory allocation strategy used by the allocator:
 - `first` – First Fit allocation
 - `best` – Best Fit allocation
 - `worst` – Worst Fit allocation
 - `buddy` – Buddy System allocation
- The selected strategy remains active until explicitly changed.

Memory Allocation

```
malloc <size>
```

- Allocates a block of memory of the requested size (in bytes) using the currently selected allocation strategy. Each successful allocation is assigned a **unique block ID**, which is later used for deallocation

Memory Deallocation

```
free <id>
```

Frees the memory block associated with the given block ID.

The allocator performs block coalescing where applicable to reduce fragmentation.

▪Memory Layout Inspection

```
dump memory
```

Prints the current state of the heap, showing:

- Block address ranges
- Allocation status (USED / FREE)
- Block sizes (including metadata)
- This command is primarily used for debugging and visualization of allocator behavior.

Allocation Statistics

stats

Displays allocation-related statistics, including:

- Total allocation requests
- Successful allocations
- Failed allocations
- These statistics help evaluate allocator performance under different workloads.

Fragmentation Analysis

Frag

Computes and displays fragmentation statistics for the heap, providing insight into external fragmentation caused by allocation patterns.

Exit CLI

exit

Terminates the CLI session and exits the simulator.

Limitations

- This project is a **functional memory management simulator** with a deliberately limited scope to ensure clarity and correctness.
- **No Virtual Memory Support**
The simulator operates directly on a single contiguous physical heap. Paging, page tables, TLBs, and address translation are not implemented.
- **Metadata Overhead (~40 Bytes per Block)**
Each memory block stores inline metadata inside the heap, resulting in a fixed overhead of approximately 40 bytes per allocation. This causes internal fragmentation, especially for small allocations.
- **Single-Threaded Execution**
The allocator and cache are not thread-safe. No concurrency or synchronization mechanisms are implemented.
- **Buddy Allocator Constraints**
The buddy system uses power-of-two block sizes, which can lead to internal fragmentation. Heap visualization tools are limited for this allocator.
-

Future Work

- The simulator can be extended in several directions:
- **Virtual Memory Integration**
Add paging, page tables, and virtual-to-physical address translation.
- **Advanced Cache Policies**
Implement LFU replacement, write-back caches, and coherence protocols.
- **Concurrency Support**
Introduce thread-safe allocation and simulate multi-threaded workloads.
- **Reduced Metadata Overhead**
Explore external metadata storage or optimized block layouts.
- **Enhanced Visualization and Analysis**
Add graphical heap views and detailed cache access tracing.

Allocation and Deallocation Behavior (CLI-Based Demonstration)

▪ This section demonstrates the behavior of different allocation strategies using the interactive command-line interface (CLI). The goal is to visually verify how memory blocks are allocated, freed, and reused when the allocation strategy is changed at runtime.

▪ The CLI is launched using:

```
.\memsim.exe --cli
```

▪ All experiments below are performed on a heap of size **1024 bytes**.

Step 1: Heap Initialization and First Fit Allocation

▪ Commands executed:

```
init memory 1024
```

```
strategy first
```

```
malloc 100
```

```
malloc 200
```

```
dump memory
```

Explanation:

▪ The heap is initialized with a contiguous block of 1024 bytes.

▪ The allocation strategy is set to **First Fit**.

▪ Two allocations are requested: 100 bytes and 200 bytes.

▪ Each allocation internally consumes additional metadata (≈ 40 bytes per block), so the actual occupied sizes become:

- 100 $\rightarrow$ 140 bytes
- 200 $\rightarrow$ 240 bytes

▪ First Fit scans the heap linearly and places each allocation in the first available free block large enough to satisfy the request.

▪ The remaining memory stays as a single free block at the end of the heap.

```
PS C:\Users\saral\OneDrive\Desktop\memory-simulator> .\memsim.exe --cli
Memory Simulator (memsim)
> init memory 1024
Memory initialized with size 1024
> strategy first
Strategy set to FIRST FIT
> malloc 100
Allocated block id=1 at address=000002BE2B385018
> malloc 200
Allocated block id=2 at address=000002BE2B3850A4
> dump memory

===== MEMORY DUMP =====
[0x2b384ff0 - 0x2b38507b] USED (140 bytes)
[0x2b38507c - 0x2b38516b] USED (240 bytes)
[0x2b38516c - 0x2b3853ef] FREE (644 bytes)
```


Step 2: Deallocation and Free Block Creation

Commands executed:

```
free 1
```

```
dump memory
```

Explanation:

- Block with ID 1 (the 100-byte allocation) is freed.
- This creates a **free hole** at the beginning of the heap.
- No coalescing occurs yet because adjacent blocks are still in use.
- The heap now consists of:
 - One free block (140 bytes)
 - One used block (240 bytes)
 - One large free block at the end

```
PS C:\Users\saral\OneDrive\Desktop\memory-simulator> .\memsim.exe --cli
Memory Simulator (memsim)
> init memory 1024
Memory initialized with size 1024
> strategy first
Strategy set to FIRST FIT
> malloc 100
Allocated block id=1 at address=000002BE2B385018
> malloc 200
Allocated block id=2 at address=000002BE2B3850A4
> dump memory

===== MEMORY DUMP =====
[0x2b384ff0 - 0x2b38507b] USED (140 bytes)
[0x2b38507c - 0x2b38516b] USED (240 bytes)
[0x2b38516c - 0x2b3853ef] FREE (644 bytes)
> free 1
Block 1 freed
> dump memory

===== MEMORY DUMP =====
[0x2b384ff0 - 0x2b38507b] FREE (140 bytes)
[0x2b38507c - 0x2b38516b] USED (240 bytes)
[0x2b38516c - 0x2b3853ef] FREE (644 bytes)
```

Step 3: Switching Strategy and Allocation Using Best Fit

Commands executed:

```
strategy best
```

```
malloc 90
```

```
dump memory
```

Explanation:

- The allocation strategy is switched to **Best Fit** without reinitializing the heap.
- A request for 90 bytes is made.
- Best Fit scans **all free blocks** and selects the **smallest free block that can satisfy the request**.
- The previously freed 140-byte block is chosen instead of the larger 644-byte block.
- This demonstrates that strategy switching affects **placement decisions**, not heap structure.

Fragmentation Analysis

- Total Used Memory:** Sum of all allocated blocks (including metadata)
- Total Free Memory:** Sum of all free blocks
- Internal Fragmentation:** Difference between allocated block size and requested size (e.g., $140 - 130 = 10$ bytes)
- Largest Free Block:** Largest contiguous free region available
- External Fragmentation:** Computed as:
$$\text{External Fragmentation} = 1 - \frac{\text{Largest Free Block}}{\text{Total Free Memory}}$$
- Since most free memory is still contiguous, external fragmentation remains low or zero.

```
Memory Simulator (memsim)
> init memory 1024
Memory initialized with size 1024
> strategy first
Strategy set to FIRST FIT
> malloc 100
Allocated block id=1 at address=000001C403915018
> malloc 200
Allocated block id=2 at address=000001C4039150A4
> dump memory

===== MEMORY DUMP =====
[0x03914ff0 - 0x0391507b] USED (140 bytes)
[0x0391507c - 0x0391516b] USED (240 bytes)
[0x0391516c - 0x039153ef] FREE (644 bytes)
> free 1
Block 1 freed
> dump memory

===== MEMORY DUMP =====
[0x03914ff0 - 0x0391507b] FREE (140 bytes)
[0x0391507c - 0x0391516b] USED (240 bytes)
[0x0391516c - 0x039153ef] FREE (644 bytes)
> strategy best
Strategy set to BEST FIT
> malloc 90
Allocated block id=3 at address=000001C403915018
> dump memory

===== MEMORY DUMP =====
[0x03914ff0 - 0x0391507b] USED (140 bytes)
[0x0391507c - 0x0391516b] USED (240 bytes)
[0x0391516c - 0x039153ef] FREE (644 bytes)
> frag
```

```
----- FRAGMENTATION STATS -----
Total used memory      : 380 bytes
Total free memory      : 644 bytes
Allocated blocks       : 2
Internal fragmentation : 10 bytes
Largest free block     : 644 bytes
External fragmentation : 0.000
Allocator overhead     : 80 bytes
-----
> █
```

Step 3: Switching Strategy and Allocation Using Worst Fit

Commands executed:

```
strategy worst
```

```
malloc 90
```

```
dump memory
```

Explanation:

- The allocation strategy is switched to **Worst Fit** without reinitializing the heap.
- A request for 90 bytes is made.
- Worst Fit scans **all free blocks** and selects the **largest free block that can satisfy the request**.
- The previously freed 644 bytes block is chosen instead of the smaller 140-byte block.
- This demonstrates that strategy switching affects **placement decisions**, not heap structure.

Fragmentation Analysis

- Total Used Memory:** Sum of all allocated blocks (including metadata)
- Total Free Memory:** Sum of all free blocks
- Internal Fragmentation:**
Difference between allocated block size and requested size : 0 bytes
Largest Free Block: Largest contiguous free region available
- External Fragmentation:**
Computed as:
$$\text{External Fragmentation} = 1 - \frac{\text{Largest Free Block}}{\text{Total Free Memory}}$$
- Using the above formula external fragmentation comes out to be 0.214

```
PS C:\Users\saral\OneDrive\Desktop\memory-simulator> .\memsim.exe --cli
Memory Simulator (memsim)
> init memory 1024
Memory initialized with size 1024
> strategy first
Strategy set to FIRST FIT
> malloc 100
Allocated block id=1 at address=000002BE2B385018
> malloc 200
Allocated block id=2 at address=000002BE2B3850A4
> dump memory
```

```
===== MEMORY DUMP =====
[0x2b384ff0 - 0x2b38507b] USED (140 bytes)
[0x2b38507c - 0x2b38516b] USED (240 bytes)
[0x2b38516c - 0x2b3853ef] FREE (644 bytes)
> free 1
Block 1 freed
> dump memory
```

```
===== MEMORY DUMP =====
[0x2b384ff0 - 0x2b38507b] FREE (140 bytes)
[0x2b38507c - 0x2b38516b] USED (240 bytes)
[0x2b38516c - 0x2b3853ef] FREE (644 bytes)
> strategy worst
Strategy set to WORST FIT
> malloc 90
Allocated block id=3 at address=000002BE2B385194
> dump memory
```

```
===== MEMORY DUMP =====
[0x2b384ff0 - 0x2b38507b] FREE (140 bytes)
[0x2b38507c - 0x2b38516b] USED (240 bytes)
[0x2b38516c - 0x2b3851ed] USED (130 bytes)
[0x2b3851ee - 0x2b3853ef] FREE (514 bytes)
> frag
```

```
----- FRAGMENTATION STATS -----
Total used memory      : 370 bytes
Total free memory      : 654 bytes
Allocated blocks       : 2
Internal fragmentation : 0 bytes
Largest free block     : 514 bytes
External fragmentation : 0.214
Allocator overhead     : 80 bytes
-----
```

Coalescing

This snippet from the cli shows coalescing of 140-bytes and 240-bytes to a single 380 sized block

```
PS C:\Users\saral\OneDrive\Desktop\memory-simulator> .\memsim.exe --cli
Memory Simulator (memsim)
> init memory 1024
Memory initialized with size 1024
> malloc 100
Allocated block id=1 at address=0000023441615018
> malloc 200
Allocated block id=2 at address=00000234416150A4
> malloc 300
Allocated block id=3 at address=0000023441615194
> free 1
Block 1 freed
> free 2
Block 2 freed
> dump memory

===== MEMORY DUMP =====
[0x41614ff0 - 0x4161516b] FREE (380 bytes)
[0x4161516c - 0x416152bf] USED (340 bytes)
[0x416152c0 - 0x416153ef] FREE (304 bytes)
> 
```

Cache Hierarchy Simulation Test

- To validate the multi-level cache implementation, we executed a controlled simulation using memsim.exe. The test was designed to verify three key behaviors: **Cold Misses**, **L1 Hits** (temporal locality), and **Inter-level Eviction** (inclusive behavior).
- The test configuration utilized the following hierarchy:
 - **L1 Cache:** 1KB, 4-Way Associative (Smallest, fastest)
 - **L2 Cache:** 2KB, 8-Way Associative (Larger victim cache)
 - **L3 Cache:** 4KB, 16-Way Associative (Largest backing store)

```
=== CACHE HIT / MISS TEST ===
```

```
=== L1 Statistics ===
```

```
Policy      : LRU
Hits        : 4
Misses      : 12
Total       : 16
Hit Rate    : 25.00%
```

```
=== L2 Statistics ===
```

```
Policy      : LRU
Hits        : 4
Misses      : 8
Total       : 12
Hit Rate    : 33.33%
```

```
=== L3 Statistics ===
```

```
Policy      : LRU
Hits        : 0
Misses      : 8
Total       : 8
Hit Rate    : 0.00%
```

Analysis of Output Statistics

▪The simulation generated the following statistics (refer to **Figure 1** above):

Layer 1 (L1) Analysis

- Total Accesses:** 16
- Hits:** 4
- Misses:** 12
- Hit Rate:** 25.00%

Explanation: The low hit rate is intentional. The test performed 16 total operations across four phases.

4 Cold Misses: The first 4 requests (Phase 1) were for empty data, resulting in compulsory misses.

4 Hits: We immediately re-accessed the same data (Phase 2), resulting in 4 Hits.

4 Capacity Misses: We accessed 4 *new* blocks (Phase 3). Since L1 is only 4-way associative, these new blocks forced the eviction of the original data.

Conflict Misses: We requested the original data again (Phase 4). Since it had been evicted to make room for the new data, L1 missed again.

Layer 2 (L2) Analysis

- Total Accesses:** 12
- Hits:** 4
- Misses:** 8
- Hit Rate:** 33.33%

Explanation: L2 only receives requests that *miss* in L1.

Why 12 Accesses? L1 missed 12 times, passing those requests down to L2.

Why 4 Hits? This is the critical success metric. In Phase 4, when L1 missed on the original data (because it was evicted), the request dropped to L2. Since L2 is **8-way associative** (larger capacity), it had retained the data that L1 discarded. This confirms the **Victim Cache** behavior is functioning correctly.

Layer 3 (L3) Analysis

- Total Accesses:** 8
- Hits:** 0
- Misses:** 8
- Hit Rate:** 0.00%

Explanation: L3 only receives requests that miss in *both* L1 and L2.

The 8 accesses correspond to the 8 unique blocks of data requested throughout the entire program (4 in Phase 1, 4 in Phase 3).

Since all 8 blocks were being accessed for the very first time in the program's lifecycle, they were **compulsory misses** (Cold Misses) fetched directly from main memory. This 0% hit rate is expected for a test that never re-visits data old enough to have been pushed all the way to L3.

4.3 Conclusion

The results confirm that the cache controller correctly manages data movement:

Hits occur when data is reused immediately (L1).

Evictions occur when set associativity is exceeded.

Inclusion is maintained, as L2 successfully served data that was evicted from L1, preventing a costly main memory access.